

Problem 1: Geometric Image Modification

I. Motivation

Our goal for this problem is to apply a spatial warping technique that takes a square image (Bird & Cat) and transforms it into a star-shaped output. We will then apply the reverse and revert the star-transformed image back into the original image. The main reason for exploring this behavior is because geometric transformations allow for images to be manipulated while preserving their initial visual content. Some real-world applications of this include image morphing, creating visual effects, and computer graphics. When we apply our forward warping, we explore how the coordinates of the pixels in the original image can be mapped into new positions and how spatial distortion can affect different regions of an image. After we applied the forward warp, we introduced reverse warping which mapped each pixel back to the corresponding location in the input image. With reverse warping, we explore how the recovered or reverted image matches the initial input image, highlighting any distortions that may come to light due to us applying two transformations.

II. Approach

Our approach for this problem first reads the raw RGB image and we store the values of the pixels in a 1-D array. We center the image at (cx, cy) with each pixel expressed in the coordinates (x, y) . Now for each of the outputted pixels, we compute the polar angle with the formula below:

$$\theta = \text{atan2}(y, x)$$

Also we use a scaling factor with my starShift function which has the scale set to:

$$1 - \beta \cos^2(2\theta) \qquad \beta = \frac{\text{arc}}{\text{halfSize}}$$

This scaling factor is between 0 and 1 and it impacts how much we will shrink the image based on a given angle (this case: theta). The beta in this case controls how strong the distortion is. And we clamp this value to below 1 so the scale can be positive. In other words, the scaling factor will shrink the image stronger along the vertical and horizontal axes instead of along the diagonals which yields our star distortion. Once we do this, we see if a pixel is within the star

boundary using the Chebyshev distance condition. If we see that a pixel is within a valid region, then we apply backward mapping to locate the corresponding position in the original square image by dividing the centered coordinate by our scale factor. The thickness of the black arc region was set to 128 to adhere to the homework requirements.

Now to revert the image to the initial square image, we first take each pixel in the transformed image and apply the polar angle and the scale factor. Then we multiply the centered coordinates by the scale factor to get the position in the star-shaped image. We use bilinear interpolation to estimate the RGB value for fractional coordinates. Now, for pixels that were matched into the black arc regions of the image, they cannot yield any valid color information, so those regions are slightly distorted since they can't be fully reconstructed.

III. Results

Bird Images:

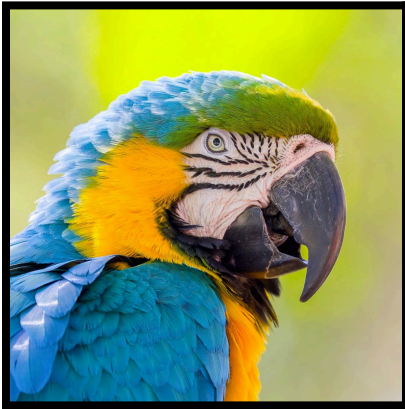


Figure 1: Bird.png



Figure 2: Bird_star.png



Figure 3: Bird_revert.png

Cat Images:



Figure 4: Cat.png



Figure 5: Cat_star.png



Figure 6: Cat_revert.png

IV. Discussion

In our results section, we can see that Figures 2 and 5 have successfully warped Bird and Cat images into star-shaped outputs. We can see that both figures have sought out transformations with the black regions clearly visible across the horizontal and vertical axes while preserving the diagonal regions of both images. This is due to the $\cos^2(2\theta)$ term, which is zero on the diagonals yielding our scale factor to approach 1, causing a minimal distortion in those diagonal directions. Conversely, however, across the vertical & horizontal axes, we have scaling factors that reach small values, which in turn pull the image inward, creating a curved black arc region we see in the images.

In our reverse warped images (Figures 3 & 6), we can see that recovered images are very similar to the original images (Figure 1 & 4) with the colors and overall structure being preserved. Despite this similarity, we do spot a couple of small differences making the two images different. One issue is the slight blurring of the image across the image. Because the bilinear interpolation is being used when we sample the pixel values at coordinates of fractional values, the pixel intensities are estimated by blending neighboring pixels. This introduces a slight smoothing effect that reduces the sharpness of the overall image. We can spot this in the feathers of the left side of the bird in Figure 3 or in the base of the paw (towards the right) in Figure 6 as both of those regions are slightly more blurry than their initial images.

Another slight difference can be seen when comparing Figure 1 and Figure 3 of the bird image. We can see that there is a slight loss of detail in the bottom left corner of the reconstructed image. This can be attributed to the fact that some pixels are mapped outside the star shape region and become black to be added to the black arc. Due to this, these pixels do not have valid color information so when we revert the image, we struggle to fully recover them. This is why when we look at the fine details of the feathers in that bottom left region, we can see that there is a slight degradation in that region. This highlights how images are not fully invertible and that not every pixel can be recovered if lost during the forward warping process.

Problem 2: Homographic Transformation and Image Stitching

I. Motivation

In this problem we are investigating image stitching and homographic transformation by taking in multiple overlapped images into a single panoramic image. Due to the real world constraints of imaging, a single image is not able to capture the entire field of view of a scene. Thus, through image stitching, we can align and merge multiple images from various viewpoints and merge it into a single continuous image. This is commonly used across computer vision and mobile photography. We specifically explore how corresponding control points across overlapping images can be estimated homographies and how local feature descriptors (SIFT) can assist in automating this process. We will be applying these techniques to Broad images and seeing how blending and geometric alignment can build a panorama.

II. Approach

Our approach for this problem includes us first using SIFT (Scale-Invariant Feature Transform) to establish the correspondences across overlapping images. We took each image and converted it into grayscale. Then SIFT was applied to detect up to 3000 keypoints and calculate their descriptors. After that, feature matching between the image pairs was done with a k nearest neighbor matcher with k set to 2. Now to remove any ambiguous matches, we used a ratio test with a threshold value of 0.75 so best matches were only kept if the best match distance was less than 75% of the 2nd best match.

From this approach, our program yields 294 matches for the left-middle image pair and 260 matches for the right-middle pair as seen below.

```
Panorama Size: 1440 x 812
Left-Middle matches: 294, inliers: 242
Right-Middle matches: 260, inliers: 167
```

After we got the initial matches, RANSAC (Random Sample Consensus) was used to estimate the homography and filter out any incorrect matches that could have arisen from SIFT matches. During each iteration, 4 random pair points were chosen and used to estimate a homography by solving an 8×8 linear system with H_{33} fixed to 1. Next we calculated the reprojection error for all matches and we set any match with an error below 3 pixel was listed as an inlier. After we ran 3000 iterations, we got 242 inliers for left-middle matches and 167 inliers for right-middle matches as seen above.

We next refine the final homography by using least squares over the least squares across OpenCV's SVD solver. We constructed the panorama with the middle image being the reference frame with an identity homography. The next four corners of each image were then warped with their respective homographies to get the max and min coordinates of the full canvas. After that, we applied a translation matrix to shift all of the coordinates so the panorama image starts at the origin (0,0). Next, we wrap each image into the canvas by taking each output pixel and inverting the homography to get the corresponding source location and bilinear interpolation to get the sample pixel value. For regions where the images would overlap, values of pixels were accumulated into a sum and count buffer and averaged to get our final panorama output.

III. Results

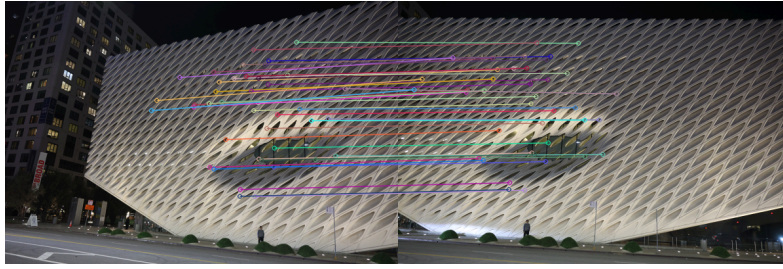


Figure 1: Left-Middle Matching

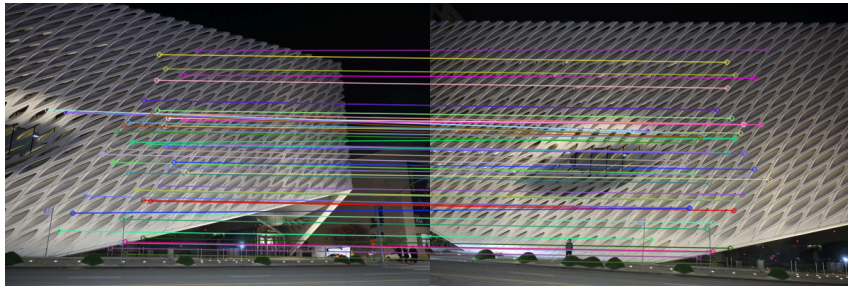


Figure 2: Right- Middle Matching

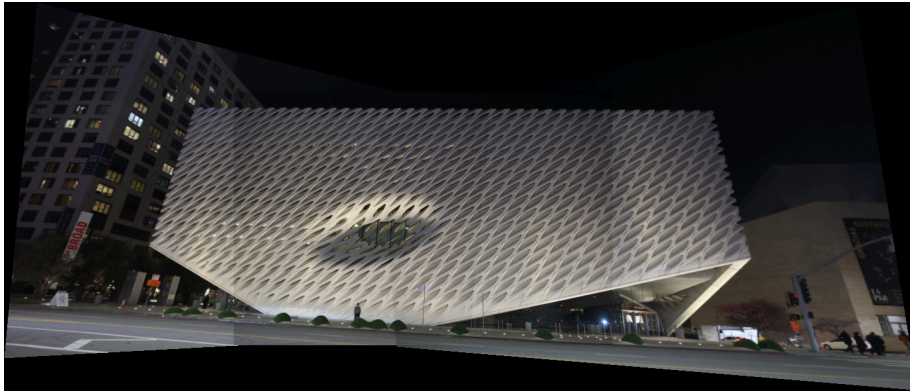


Figure 3: Final Output Panorama

Index	SrcX	SrcY	DstX	DstY
8	223.23	166.23	24.23	134.58
12	228.70	234.02	24.63	212.36
14	229.64	132.24	35.94	97.18
15	230.95	344.63	16.26	337.62
16	231.30	150.85	36.37	118.51
17	236.07	65.93	51.98	23.02
19	240.86	192.68	44.39	166.74
21	247.03	149.27	56.36	119.26
23	256.82	132.38	70.19	102.13
24	256.96	283.65	54.71	269.58
28	265.09	303.39	62.11	291.78
29	266.53	115.86	83.37	85.99
30	266.79	144.58	81.08	117.10
32	268.58	333.36	63.76	324.41
35	277.89	125.71	96.02	98.56
36	278.51	178.33	91.27	155.88

Figure 4: Left Middle points (20 points)

Index	SrcX	SrcY	DstX	DstY
1	5.57	181.23	245.04	212.17
5	12.57	172.44	250.74	203.52
6	13.34	122.79	247.84	161.03
7	15.53	218.72	256.05	243.54
10	20.77	178.06	256.41	208.15
12	23.82	106.40	254.08	145.97
13	29.24	133.84	260.24	168.88
14	32.71	65.38	257.82	109.42
15	34.66	183.14	267.71	211.40
17	35.76	289.01	276.83	304.33
20	41.60	187.80	273.89	214.75
21	44.59	362.12	289.59	370.03
22	45.06	184.99	275.82	211.66
23	46.04	327.50	288.44	339.20
27	48.58	236.42	283.18	257.03
28	50.74	354.05	293.71	363.10
29	50.88	48.17	270.49	91.51
30	51.21	188.84	281.64	214.56
31	51.96	296.45	290.48	310.99

Figure 5: Right Middle points (20 points)

IV. Discussion

For this problem, control points were detected using the SIFT feature detector. For this problem we were asked to have a minimum of four control point pairs. But since using only 4 pairs makes the system sensitive to noise, we used much more. With the SIFT detector, it identified key points in each image that were invariant to scale. Then descriptors were matched between overlapping images with a knn matcher and a Lowe's ratio test with threshold of 0.75 was applied so that we only use reliable correspondences. This yielded our matches of 294 for left and middle and 260 for right-middle. This was not our final number as SIFT could have incorrect correspondences so RANSAC was applied. In each iteration 4 random point pairs were used to compute a homography and their reprojection error was computed to evaluate all matches. Any points with error less than 3 were considered inliers. After 3000 iterations, we got our final values (**242 inlier for left-middle pairs and 167 inlier for right middle pairs**). This was then used to compute the final homography using least-squares. With all these estimated homographies, left and right images were warped into the system. The final panorama was constructed with all the warped images into a canvas and averaging pixel values in overlapping regions. Our result Figure 3 shows the merged output which gives a strong look at what an ideal image with all sides added. We do want to emphasize that Figures 4 and 5 show a subset of the pairs (20) to give an idea of the correspondences used for estimating the homography.

Problem 3: Morphological processing

I. Motivation

Problem 3(a): Basic morphological process implementation:

The main goal for this part is to reduce our binary pattern into a minimally thick or thin representation that maintains the image's connectivity and structure. The significance of this approach is that one-pixel-wide skeletons can retain the key shape of an object's topology while removing any excess boundary pixels. This is commonly used in pattern matching, structural analysis, and shape recognition. In this problem, we apply our thinning algorithm to images of a Jar, Moon, Star, and Spring. We apply the thinning to remove all boundary pixels and to give us the final skeleton structure. From here, we can see how the thinning process can transform the thick initial binary shape into a skeleton that preserves the connectivity of an image or object.

Problem 3(b): Shape detection and counting:

In this problem, we are exploring the significance of trying to analyze and classify individual objects. The main reason for this is that even though thinning in problem 3a showed us the skeletal results, it does not provide information on the geometric characters or individual objects found in the object. The goal here is to run object analysis on the binary image, counting the various shapes. Through this approach, we can see key characteristics such as area, bounding boxes, and internal holes, which can give us key information about each object. These bits of classification allow us to distinguish across different types of shapes (like circle & rectangles), allowing us to get key stats describing the objects in the image.

II. Approach

Problem 3(a): Basic morphological process implementation:

In this problem, we first take the gray scale pattern images and convert them into binary images. Each image was binarized with a threshold = to half the maximum intensity value in that image. If a pixel intensity was greater than the 0.5 Fmax, then it was assigned a value of 1 else it would be assigned a value of 0. This outputted our binary input for our thinning algorithm.

With this binary input, the thinning algorithm uses an iterative 2 subiteration process founded upon the 8-neighborhood of each pixel. For each pixel, its eight surrounding neighbors were examined in the sequences from p2 -> p9. The first measure computed from this neighborhood, was the number of nonzero neighbors. This was to ensure that the pixel was not overly surrounded or isolated. The other measure computed was the number of transitions (from 0 to 1) in the neighborhood sequenced. This was used to determine if a pixel is laying on a boundary and if deleting the pixel would preserve connectivity.

During each full iteration, two subiterations were performed. In the first subiteration, the pixel was only marked for deletion if it had between 2-6 nonzero neighbors, exactly one 0 to 1 transition in the neighborhood, and if the directional product $(p2*p4*p6) \& (p4*p6*p8) = 0$. The significance here is that this procedure prevented the removal of pixels that are key in maintaining the correct structure of the object. The second subiteration follow a similar process but the final condition was $p2*p4*p8$ and $p2*p6*p8 = 0$. After each of the subiterations, all pixels that were marked to be deleted were removed simultaneously.

This process was repeated until convergence (when no further changes can be done in a binary image in a full iteration). For this problem specifically, we saved the intermediate iteration (20) and the final converged image so we can view how a thick binary pattern transitions into a thin skeleton structure. Thus, following the requirements of the assignment, we need to implement our own thinning algorithm from scratch.

Problem 3(b): Shape detection and counting:

In this problem, we first convert our raw input image into a binary image. Then we follow a similar procedure to Problem 3a, where any pixel with an intensity greater than $0.5 * f_{max}$ was assigned a value of 1, while the rest was assigned a value of 0. Now, to calculate the total number of white objects, we used connected-component labeling with 4-connectivity being applied to the binary image. From each unlabeled foreground pixel, we used a flood fill process that identified all pixels that belonged to the connected object. The total number of components gave us the total number of white shapes in the image. Now for the hole calculation, the Euler number was computed for all 2-by-2 pixel patterns in the binary image. The 3 types of patterns were counted with quads that have exactly one pixel (n_1), quads having two diagonally connected pixels (n_d), and quads with exactly 3 foreground pixels (n_3). The Euler number calculated is shared below:

$$E = \frac{n_1 - n_3 + 2n_d}{4}$$

Now because Euler's number satisfied the $E = \text{object} - \text{holes}$ formula, the total number of holes was calculated by $\text{holes} = \text{object} - e$. Finally, to classify as a rectangle or a circle, we computed the fill ratio, which was defined as the area of the object divided by the area of the bounding box. Now rectangles occupy most of the bounding boxes so they have fill ratios closer to 1, while the circular objects have small ratios (around $\pi/4$). As a result, I used a threshold of 0.82, where objects equal to or below it were circles, while the rest were rectangles. For this problem I would like to emphasize that two morphological properties were used Euler's number (to count holes) & a custom connected component from scratch (to count/classify objects), adhering to the homework rule which allows us to add any method as long as we don't use a library.

III. Results

Problem 3(a): Basic morphological process implementation:

Spring Images:

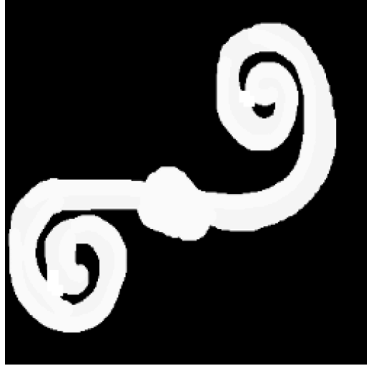


Figure 1: Original Spring

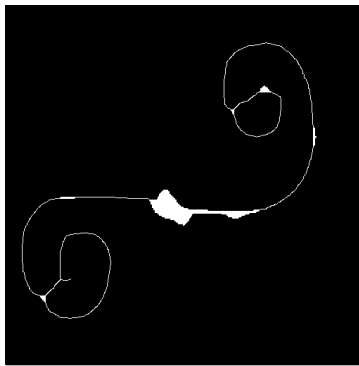


Figure 2: Thin (20 iteration)

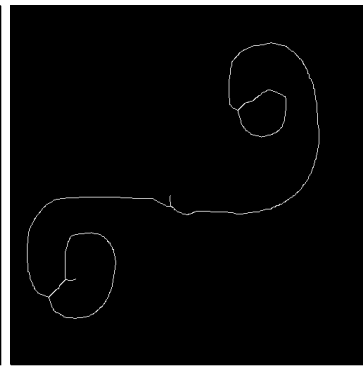


Figure 3: Final Thin Output

Star Image:



Figure 4: Original Star

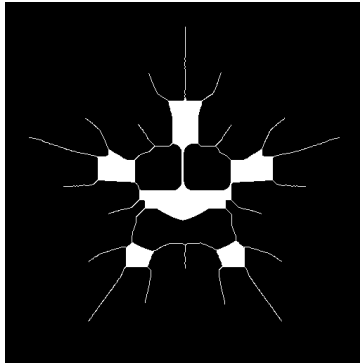


Figure 5: Thin (20 iteration)

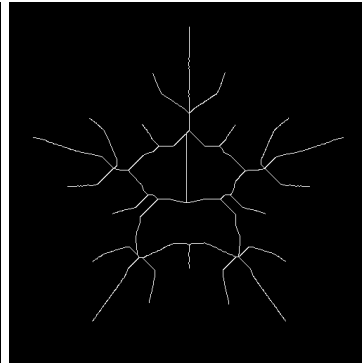


Figure 6: Final Thin Output

Jar Image:



Figure 7: Original Jar

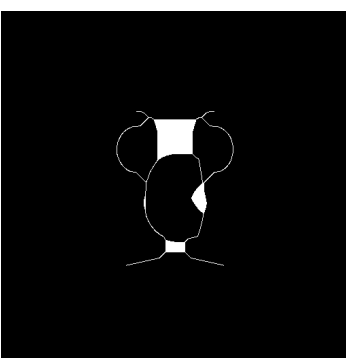


Figure 8: Thin (20 iteration)

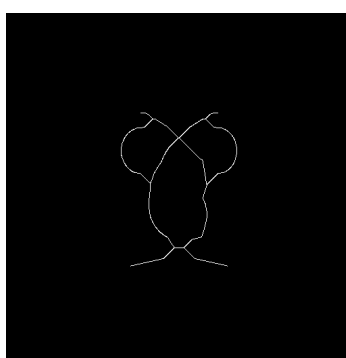


Figure 9: Final Thin Output

Moon Image:

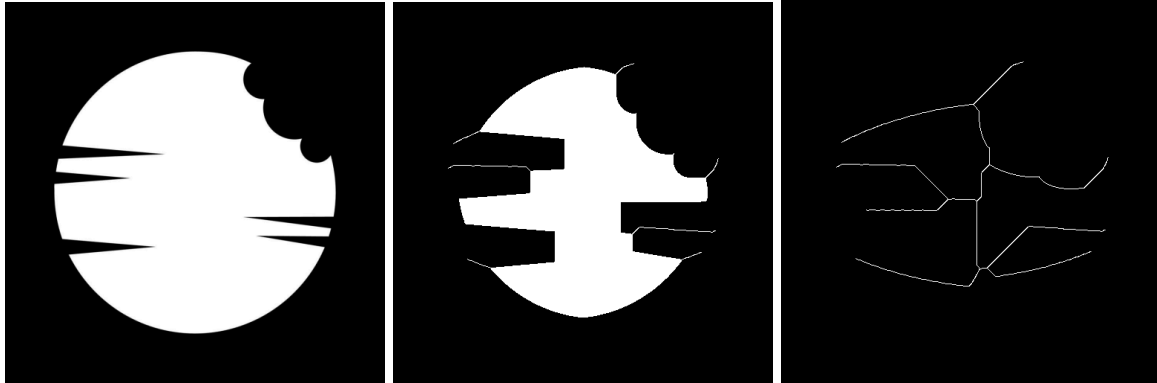


Figure 10: Original Moon

Figure 11: Thin (20 iteration)

Figure 12: Final Thin Output

Problem 3(b): Shape detection and counting:

Metric	Count
Total number of holes	9
Total number of white shapes	14
Total number of white rectangles	7
Total number of white circles	7

Figure 13

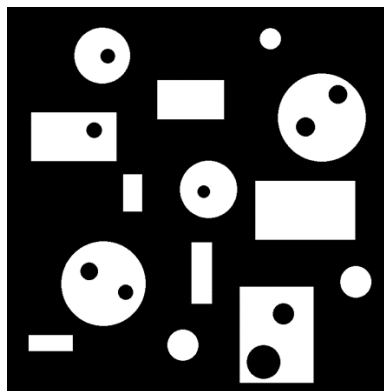


Figure 14: Board Image

IV. Discussion

Problem 3(a): Basic morphological process implementation:

The thinning operation in my own words is an iterative process that removes pixels, specifically boundary pixels from a binary object. The main goal here is to preserve connectivity and the overall structure so we can reduce a thick shape into a one pixel wide skeleton which would be the topography of the object. In my implementation, we looked at each foreground pixel against its neighborhood and saw if the two conditions were passed (# of non zero neighboring pixels & # of 0 to 1 transitions). These were done to ensure the boundary pixels that are detected only if they do not prevent the disconnection of the object. Furthermore, the thinning algorithm ran two sub iterations for each 1 iteration which ensured that pixels that satisfied the given thinning conditions were marked and removed. This process continues until no pixels can be removed in which we have finally reached convergence.

For the images, in Figures 1-3 we can see that the Spring image went from being a very thick image into a very skinny image in Figure 3. Specifically here we can see that by Figure 2 (20 iterations) we already have a very skinny outline with blobs of still thickness that are fully later removed by Figure 3. For Figures 4-6, it is interesting to see what is considered the skeleton of the star because there are many tiny edges that I would have assumed would have been deleted in this process but in fact were kept (see the edge between the top and middle right edges). In Figures 7-9, we can see the Jar preserved the circular edges which could be used to hold the Jar. But Figure 9 did remove the upper lid which is interesting to see what was considered a boundary pixel. The most interesting output was Figure 12 for the moon because the skeleton preserved more of the sharp edges instead of the overall curvature of the moon as seen in Figure 10.

Problem 3(b): Shape detection and counting:

In this problem, our results (as shown in Figure 13) were 9 holes, 14 white objects, 7 rectangles, and 7 circles. The total number of holes was 9, and this was computed with Euler's number formula (check the approach section, which breaks it down fully). Since our $e = \text{object} - \text{holes}$, we can get our holes by doing $\text{objects} - e$. This yielded our result, where the holes were listed as black circular regions within white objects. In the second question, we found the total number of white objects by using a connected component labeling 4 connectivity approach from scratch, where each distinct white region that is fully disconnected is listed as a separate object. This includes circles, rectangles, and thin bar shapes in the image.

For the total number of white rectangles (which was 7), objects were classified as rectangles when their object area divided by the bounding box was greater than 0.82, as rectangles tend to have bounding boxes closer to a fill ratio of 1, while circular objects had a lower one ($\pi/4$). The next approach to find the total number of white circles (7) was just the remaining objects left after classifying rectangles. All these values can be verified by looking at the image and manually classifying.